PROJECT BASED EXPERIENTIAL LEARNING REPORT

ON

" Smart Rice Farming System "

Under the Course

PROJECT BASED EXPERIENTIAL LEARNING

SUBMITTED BY

PRN	NAME OF STUDENT
1022512003	Rushikesh Gajanan Borade
1022512010	Shantanu Mandar Padhye
1022512013	Raghav Bholeshwar Palange
1022512028	Ayush Yogesh Bhosale

2026-27

Third Year Electronics and Communication Engineering (TY ECE)

Division: B , Semester - I

Under the Guidance Of
Dr. Hrushikesh Vanjari

Department of Electronics and Communication (AIML) Engineering
School of Engineering and Technology
DES Pune University, Pune.

Program: B. Tech Electronics and Communication Engineering
School of Engineering and Technology
DES Pune University, Pune

CERTIFICATE

This is to certify that the students **Rushikesh Gajanan Borade** (PRN 1022512003), **Shantanu Mandar Padhye** (PRN 1022512010), **Raghav Bholeshwar Palange** (PRN 1022512013) and **Ayush Yogesh Bhosale** (PRN 1022512028) are studying in **TY ECE (Division: B)** course in SEM I and they have successfully completed and submitted the **Project Based Experiential Learning Report under the Course "Project Based Experiential Learning"**, entitled **"Smart Rice Farming System"**. This study is a partial fulfilment of the degree of Bachelor of Technology in Electronics and Communication Engineering of **DES Pune University, Pune** during the academic year **2026-2027**.

Dr. Hrushikesh Vanjari
Mentor

Dr. Prachi Joshi
HoS

ACKNOWLEDGMENT

We express our sincere gratitude to our mentor, Dr. Hrushikesh Vanjari, for his continuous guidance, encouragement and valuable technical suggestions throughout the Project Based Experiential Learning course. We also thank the Head of School, Dr. Prachi Joshi, and the faculty and staff of the Department of Electronics and Communication (AIML) Engineering, School of Engineering and Technology, DES Pune University, Pune, for providing the laboratory facilities and support needed to complete this project.

We are grateful to the open-source communities and platforms behind Arduino/ESP32, Adafruit IO and Ultralytics YOLO, and to the publishers of the rice-leaf image datasets and research papers we referred to. Finally, we thank our families and classmates for their support and feedback.

ABSTRACT

Agriculture depends on careful management of water, environmental conditions and crop health. Conventional irrigation and crop monitoring are largely manual, which can lead to unnecessary water use and delayed identification of crop diseases. This project presents a **Smart Rice Farming System** that combines **Internet of Things (IoT)** based field monitoring with **Artificial Intelligence (AI)** based rice disease detection.

The IoT part is built around an **ESP32** microcontroller. It reads a soil moisture sensor, a rain sensor, a DHT11 temperature and humidity sensor, an LM35 temperature sensor and an LDR, and frequently sends these readings to the **Adafruit IO** cloud platform, where they are shown on a dashboard in real time and stored as feed history. The same ESP32 controls the irrigation pump through a relay module; the pump is switched ON or OFF by the user from the Adafruit IO dashboard, so irrigation can be done remotely while watching live field conditions.

The AI part is a **YOLO-based object detection model** trained on manually labelled rice-leaf images to detect three major rice diseases: **Rice Blast, Rice Blight and Brown Spot**. The model has been trained and currently works on leaf images independently of the IoT subsystem.

Compared with the original proposal, the microcontroller changed from ESP8266 to ESP32, the common dashboard was replaced by Adafruit IO, and threshold-based automatic irrigation was replaced by user-controlled remote irrigation. Training an AI/ML model from the data collected by the IoT system, and merging the disease-detection model with the IoT platform, are the **future enhancements** of the project.

INDEX

CHAPTER NO.		TOPIC	PAGE NO.
1		**Introduction**	**6**
	1.1	Background	6
	1.2	Need/Importance	6
	1.3	Objectives	6
	1.4	Scope	7
2		**Literature Review**	**8**
	2.1	Relevant concepts, theories	8
	2.2	Technologies related to the project	8
	2.3	Summary of similar works or case studies	9
3		**Proposed Methodology**	**10**
	3.1	System Overview	10
	3.2	Architecture / Workflow Diagram	11
	3.3	Data Collection / Input Description	12
	3.4	Algorithms / Techniques Used	13
	3.5	Tools, software, or equipment used	13
	3.6	Module-wise Implementation Plan	14
	3.7	Validation and Testing Strategy	15
4		**Results / Observations**	**17**
	4.1	Experimental / Implementation Setup	17
	4.2	Performance Metrics	18
	4.3	Comparison with Expected Results	19
	4.4	Observations and Analysis	20
	4.5	Validation and Verification with summary of findings	21
5		**Future Enhancement**	**22**
6		**Conclusion & Recommendations**	**23**
7		**References / Bibliography**	**24**
		Plagiarism Report	**25**
		Appendix: Sample Source Code	**26**

1. INTRODUCTION

1.1 BACKGROUND

Smart agriculture uses technologies such as the Internet of Things (IoT), artificial intelligence, computer vision and automation to improve crop monitoring and resource management. Rice is one of the major food crops and is affected by diseases such as Blast, Blight and Brown Spot, which reduce yield and quality when they are not identified early. Irrigation is another key part of rice cultivation; irrigating without knowing the actual soil and weather conditions can cause over-irrigation or under-irrigation.

The **Smart Rice Farming System** addresses both aspects. Its IoT subsystem monitors soil moisture, temperature, humidity, rain condition and light intensity using an ESP32 and uploads the data frequently to Adafruit IO, from where the user can also switch the irrigation pump ON or OFF. Its AI subsystem uses a trained YOLO model to detect Rice Blast, Rice Blight and Brown Spot from images of rice leaves. The original proposal planned a single integrated platform; during implementation the architecture was changed, and the two subsystems are currently separate (see Section 3.1).

1.2 NEED / IMPORTANCE

1. **Water management:** manual irrigation without field data can waste water; live soil-moisture and rain information helps the user decide when to irrigate.
2. **Remote operation:** a farmer cannot always be present at the field; cloud-based monitoring and pump control save time and effort.
3. **Early disease identification:** manual inspection is slow and subjective; an AI model can detect common rice diseases from leaf images and support earlier action.
4. **Low cost:** the system uses inexpensive sensors, an ESP32 board and free-tier tools, making it suitable for small farms and for learning.
5. **Data for the future:** the sensor data logged on the cloud can be used later to train ML models for smarter, data-driven irrigation.

1.3 OBJECTIVES

1. To build an ESP32-based IoT system for monitoring soil moisture, rain, temperature, humidity and light intensity in a rice-crop environment.
2. To send the sensor data frequently to the Adafruit IO cloud for real-time display and history.
3. To control the irrigation pump (through a relay module) remotely by the user from the Adafruit IO dashboard.

4. To manually label rice-leaf images for three diseases: Rice Blast, Rice Blight and Brown Spot.
5. To train a YOLO-based model for detecting these diseases from rice-leaf images and evaluate it using precision, recall and mAP.
6. To keep the design modular so that future versions can train an ML model on the collected IoT data and merge disease detection with the IoT dashboard.

1.4 SCOPE

Within the scope of the completed work:

- Reading soil moisture, rain, DHT11 temperature/humidity, LM35 temperature and LDR light-intensity values using an ESP32.
- Sending these values to Adafruit IO feeds at regular intervals and displaying them on an Adafruit IO dashboard.
- Switching a water pump through a relay module on the user's command from the Adafruit IO dashboard.
- Preparing a manually labelled rice-leaf image dataset and training a YOLO-based model to detect Rice Blast, Rice Blight and Brown Spot.

Outside the scope of the current phase (future enhancement): automatic threshold-based irrigation, ML model training on the collected IoT data, disease severity prediction, treatment recommendations, integration of the disease-detection model with the IoT dashboard, and automated image capture using a camera module.

Possible applications: remote irrigation control, remote field monitoring, water management, early rice disease detection, data collection for agricultural ML, and education/research on AIoT in agriculture.

2. LITERATURE REVIEW

2.1 RELEVANT CONCEPTS, THEORIES

Internet of Things (IoT) architecture

An IoT system has four layers: sensing (sensors and actuators), connectivity (Wi-Fi, MQTT/HTTP), cloud/data (storage and processing) and application (dashboards and control). In this project the sensors and relay form the sensing layer, the ESP32 with Wi-Fi forms the connectivity layer, Adafruit IO is the cloud layer, and its dashboard is the application layer.

MQTT publish / subscribe

MQTT is a lightweight messaging protocol based on a publish/subscribe model. A device publishes a value to a named topic (a feed in Adafruit IO), and any client subscribed to that topic receives it. Here the ESP32 publishes sensor values to feeds, and subscribes to the "pump" feed to receive the user's ON/OFF command.

Sensing and signal conversion

Analog sensors (soil moisture, rain, LM35, LDR) give a voltage that the ESP32's analog-to-digital converter (ADC) turns into a number (0–4095 for 12 bits). This raw value is converted to a meaningful unit, for example soil moisture in percent after calibration, or temperature in °C for the LM35, which gives 10 mV per °C. The DHT11 is a digital sensor that returns temperature and humidity directly.

Object detection and YOLO

Object detection finds objects in an image and gives each a bounding box, a class label and a confidence score. YOLO (You Only Look Once) performs detection in a single pass through a convolutional neural network, which makes it fast. Overlapping predictions are reduced with non-maximum suppression, and model quality is measured using precision, recall and mean average precision (mAP). Training usually starts from pre-trained weights (transfer learning) and is then fine-tuned on the task-specific labelled dataset.

2.2 TECHNOLOGIES RELATED TO THE PROJECT

Technology	Role in the project
ESP32	Microcontroller with built-in Wi-Fi; reads sensors, connects to Adafruit IO and drives the relay
Adafruit IO	Cloud platform providing feeds (MQTT), dashboards with gauges, charts and toggle, and data history

Technology	Role in the project
Sensors (soil moisture, rain, DHT11, LM35, LDR)	Measure the field and environmental parameters
Relay module and water pump	Electrically isolated switching of the irrigation pump
Ultralytics YOLO, Python, PyTorch	Training, evaluation and inference of the rice disease detection model
Image annotation tools	Manual labelling of disease regions in rice-leaf images

2.3 SUMMARY OF SIMILAR WORKS OR CASE STUDIES

The literature reviewed for this project (see References) broadly falls into three groups:

1. **IoT-based smart irrigation and field monitoring** systems, which use low-cost microcontrollers (Arduino/ESP family) with soil-moisture, temperature, humidity and rain sensors, and send data to a cloud or mobile application to monitor the field and operate a pump.
2. **Deep-learning based rice disease detection**, which uses convolutional networks and object detectors such as the YOLO family on rice-leaf images to classify or localise diseases such as Blast, Blight and Brown Spot.
3. **Combined AIoT systems**, which merge environmental sensing with image-based crop analysis, and show that the combination gives a more complete crop-management view than either technology alone.

Most existing systems treat irrigation monitoring and disease detection separately. This project builds both parts for rice, using an ESP32 + Adafruit IO monitoring and pump-control system and a YOLO-based disease detector, and plans to merge them as future work.

3. PROPOSED METHODOLOGY

3.1 SYSTEM OVERVIEW

The system has two subsystems. **Subsystem A (IoT)** reads the field sensors with an ESP32, sends the data frequently to Adafruit IO, and switches the pump when the user gives an ON/OFF command from the Adafruit IO dashboard. **Subsystem B (AI)** is a YOLO model trained to detect Rice Blast, Rice Blight and Brown Spot from rice-leaf images. Integration of the two is a future enhancement.

Changes from the proposed work

The major architecture changed from the proposal. The table compares the proposed work with the implementation.

Aspect	As proposed	As implemented	Status
Microcontroller	ESP8266	ESP32 (more ADC channels with 12-bit resolution, dual core, built-in Wi-Fi and Bluetooth)	Changed
Cloud / dashboard	A common software / dashboard layer for IoT and AI	Adafruit IO dashboard for IoT data and pump control	Changed
Role of the microcontroller	Collect sensor data and control irrigation according to predefined conditions	Frequently send sensor data to Adafruit IO and switch the pump on the user's command	Changed
Irrigation control	Automatic, based on soil moisture, rain and other conditions	Pump controlled by the user from the Adafruit IO dashboard	Changed (automation moved to future scope)
Sensors	Soil moisture, rain, DHT11, LM35, LDR	Same set of sensors	Retained
Historical data	Historical records of sensor and AI observations	Sensor history held in Adafruit IO feeds; using it to train an ML model is future scope	Partly done

Aspect	As proposed	As implemented	Status
AI disease detection	YOLO model for Rice Blast, Rice Blight, Brown Spot using labelled images	YOLO-based model trained to detect the same three diseases from rice-leaf images	Completed
Severity prediction and treatment recommendation	ML-based severity prediction and recommendations	Not part of the completed work	Future scope
AI + IoT integration	Both integrated through a common dashboard	AI and IoT run as two separate subsystems	Future scope
Automated image capture	Future extension (ESP32-CAM)	Not implemented	Future scope

3.2 ARCHITECTURE / WORKFLOW DIAGRAM

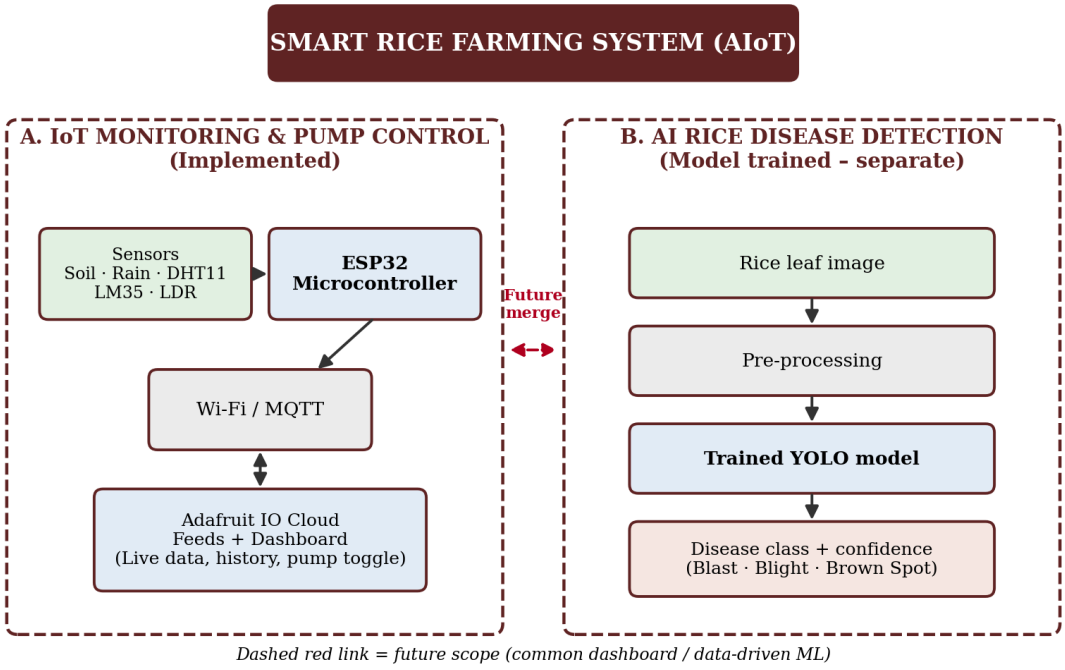


Figure 3.1: Overall System Architecture

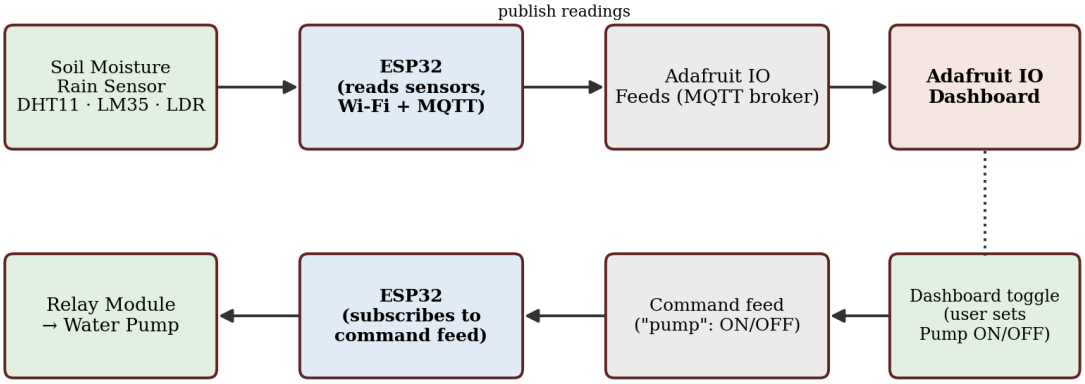


Figure 3.2: IoT Data Upload and Pump Control Flow

The upper path of Figure 3.2 is monitoring: the ESP32 reads the sensors and publishes each value to its Adafruit IO feed, from where it appears on the dashboard. The lower path is control: the user switches the pump toggle on the dashboard, the command is stored in the "pump" feed, the ESP32 (subscribed to this feed) receives it and drives the relay, which turns the pump ON or OFF.

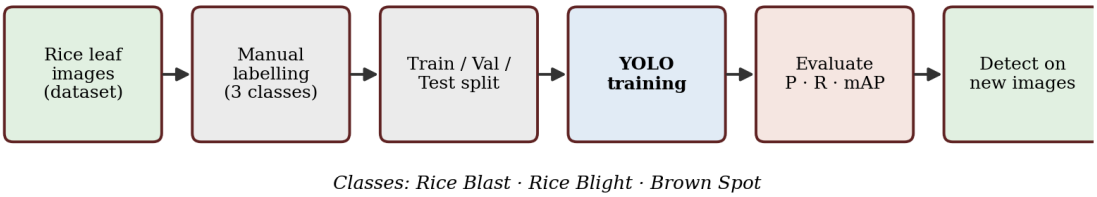


Figure 3.3: AI Model Development Workflow

3.3 DATA COLLECTION / INPUT DESCRIPTION

Sensor inputs (IoT)

Component	Parameter	Output type	Purpose
Soil Moisture Sensor	Soil moisture	Analog	Shows how wet or dry the soil is
Rain Sensor	Rain condition	Analog	Shows whether it is raining, so the user can avoid unnecessary irrigation

Component	Parameter	Output type	Purpose
DHT11	Temperature and humidity	Digital (single wire)	Environmental monitoring
LM35	Temperature	Analog (10 mV/°C)	Additional temperature measurement
LDR	Light intensity	Analog (voltage divider)	Monitors light conditions

The ESP32 reads all sensors every 10 mins and publishes each value to its own Adafruit IO feed:

Feed	Direction	Description
soil-moisture	ESP32 → Cloud	Soil moisture in percent
rain	ESP32 → Cloud	Rain sensor value / status
temperature	ESP32 → Cloud	Temperature from DHT11 (°C)
humidity	ESP32 → Cloud	Relative humidity from DHT11 (%)
lm35-temperature	ESP32 → Cloud	Temperature from LM35 (°C)
light	ESP32 → Cloud	LDR light-intensity value
pump	Cloud → ESP32	Pump ON/OFF command set by the user from the dashboard

User input

The only user input to the IoT system is the pump ON/OFF command given through the toggle block on the Adafruit IO dashboard.

Image input (AI)

Rice-leaf images were obtained from available datasets from various sources like kaggle, hugging face etc. and manually labelled for three classes: Rice Blast, Rice Blight and Brown Spot. The labelled images were divided into training, validation and testing subsets.

Subset	Number of images	Purpose
Training	1153	Learning the disease features
Validation	289	Tuning and monitoring during training

3.4 ALGORITHMS / TECHNIQUES USED

IoT firmware logic

1. Connect to Wi-Fi and Adafruit IO; subscribe to the "pump" feed; keep the pump OFF at start-up and fetch the last stored pump state.
2. Every sampling interval: read the analog sensors with the 12-bit ADC and the DHT11 digitally.
3. Convert raw values: soil moisture % = map(raw, dry value, wet value, 0, 100) after calibration; LM35 temperature (°C) = (ADC value × 3.3 / 4095) × 100.
4. Publish each converted value to its feed (skipping invalid DHT11 readings).
5. On receiving a pump message, set the relay output HIGH (ON) or LOW (OFF).

AI model

A YOLO object-detection model from the Ultralytics framework is trained on the labelled dataset, starting from pre-trained weights. For an input leaf image it outputs bounding boxes, a disease class and a confidence value. Performance is measured with:

- **Precision** = TP / (TP + FP): how many detections are correct.
- **Recall** = TP / (TP + FN): how many actual diseased regions are found.
- **mAP@0.5 and mAP@0.5:0.95:** average precision over all classes at one or several overlap (IoU) thresholds.

3.5 TOOLS, SOFTWARE, OR EQUIPMENT USED

Software

Technology	Requirement
Arduino IDE (ESP32 board package)	Programming and uploading firmware to the ESP32
C / C++ (Arduino framework)	Language for the ESP32 firmware
Adafruit IO Arduino library, DHT sensor library	Wi-Fi/MQTT connection to Adafruit IO and DHT11 reading

Technology	Requirement
Adafruit IO (cloud platform)	Feeds, dashboard (gauges, charts, toggle) and data history
Python 3	Language used for the AI model
Ultralytics YOLO (PyTorch)	Training, validation and inference of the disease-detection model
Image annotation tool	Manual labelling of Rice Blast, Rice Blight and Brown Spot images

Hardware

Hardware Component	Requirement
Microcontroller	ESP32 development board
Soil moisture sensor	Analog soil moisture sensor module
Rain sensor	Rain sensor module
Temperature and humidity sensor	DHT11
Temperature sensor	LM35
Light sensor	LDR with a fixed resistor (voltage divider)
Relay and pump	Single-channel relay module and a small DC water pump with its power supply
Power and wiring	5 V supply / USB for the ESP32, breadboard or PCB, jumper wires
Internet connection	Wi-Fi network with internet access for Adafruit IO
Computer for AI model	Laptop or desktop (GPU recommended for faster YOLO training)

3.6 MODULE-WISE IMPLEMENTATION PLAN

No.	Module	Description	Status
1	Sensor interfacing	Connect soil moisture, rain, DHT11, LM35 and LDR to the ESP32 and calibrate the readings	Completed
2	Connectivity and data upload	ESP32 firmware to connect to Wi-Fi and publish sensor values to Adafruit IO feeds at regular intervals	Completed

No.	Module	Description	Status
3	Dashboard	Adafruit IO dashboard with gauges, line charts and the pump toggle	Completed
4	Pump control	Relay module and pump switched by the ESP32 on the dashboard command	Completed
5	Dataset preparation	Collect, label and split rice-leaf images for the three diseases	Completed
6	YOLO model training and evaluation	Train the YOLO model and evaluate with precision, recall and mAP	Completed
7	ML on collected IoT data	Train a model on the logged sensor data	Future
8	AI + IoT integration	Show disease detection results on the IoT dashboard	Future

3.7 VALIDATION AND TESTING STRATEGY

Test	Method	Expected outcome
Sensor accuracy	Compare DHT11 and LM35 readings with a reference thermometer; check soil sensor in dry soil and in water	Readings close to the reference; soil % rises when wet
Data upload	Check Adafruit IO feeds and charts against the serial monitor values	Every reading appears in the correct feed at the set interval
Pump control	Toggle the dashboard switch ON and OFF several times and note the pump response	Pump follows the command each time with small delay
Connection recovery	Disconnect and restore Wi-Fi	ESP32 reconnects and resumes upload and control
Disease model	Evaluate on the held-out test images; compute precision, recall, mAP and confusion matrix; check on unseen images	Correct class with reasonable confidence; metrics reported

4. RESULTS / OBSERVATIONS

4.1 EXPERIMENTAL / IMPLEMENTATION SETUP

The IoT prototype consists of an ESP32 connected to the soil moisture, rain, DHT11, LM35 and LDR sensors and to a relay module that switches a water pump. The ESP32 connects to a Wi-Fi network and to Adafruit IO, where the feeds and the dashboard (gauges, line charts and a pump toggle) were created. The AI model was trained on a computer using the Ultralytics YOLO framework with the labelled rice-leaf dataset.

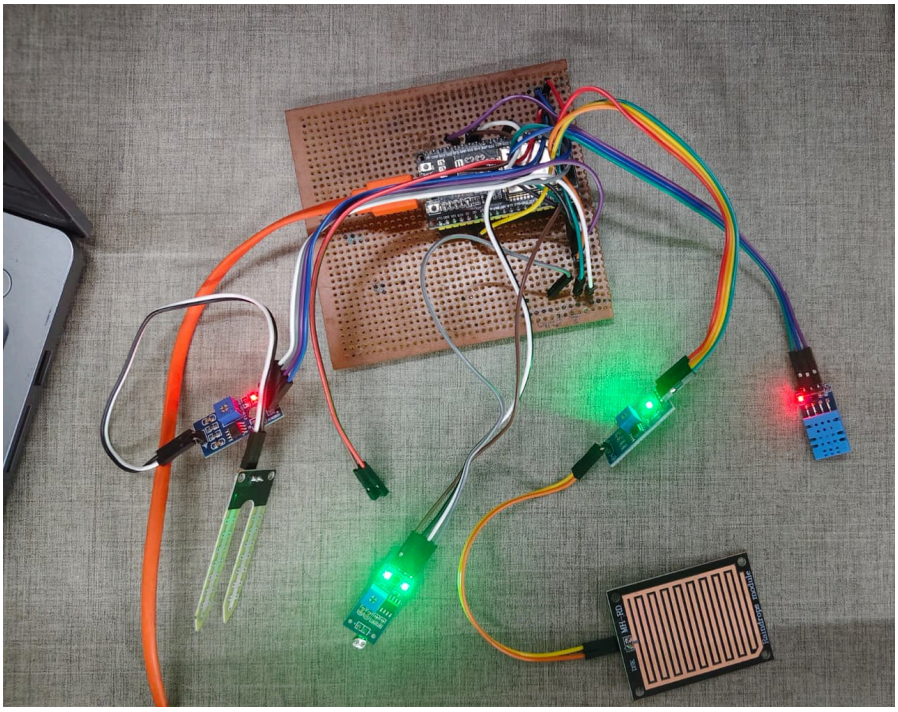


Figure 4.1: Hardware Prototype

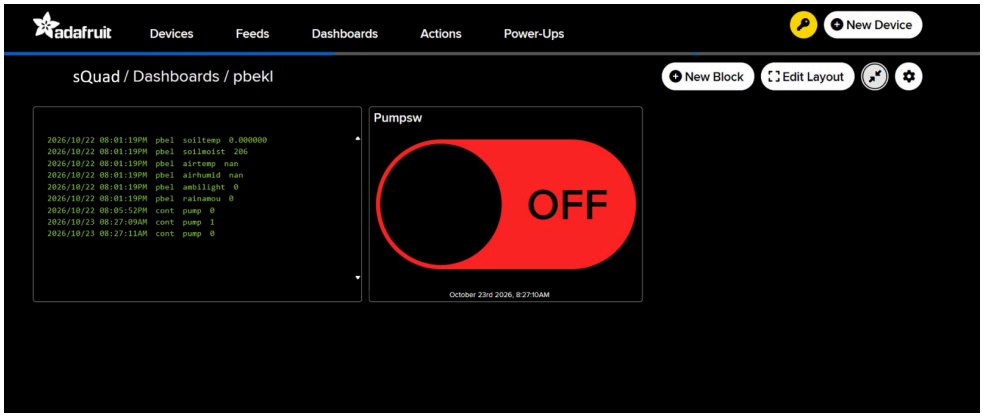
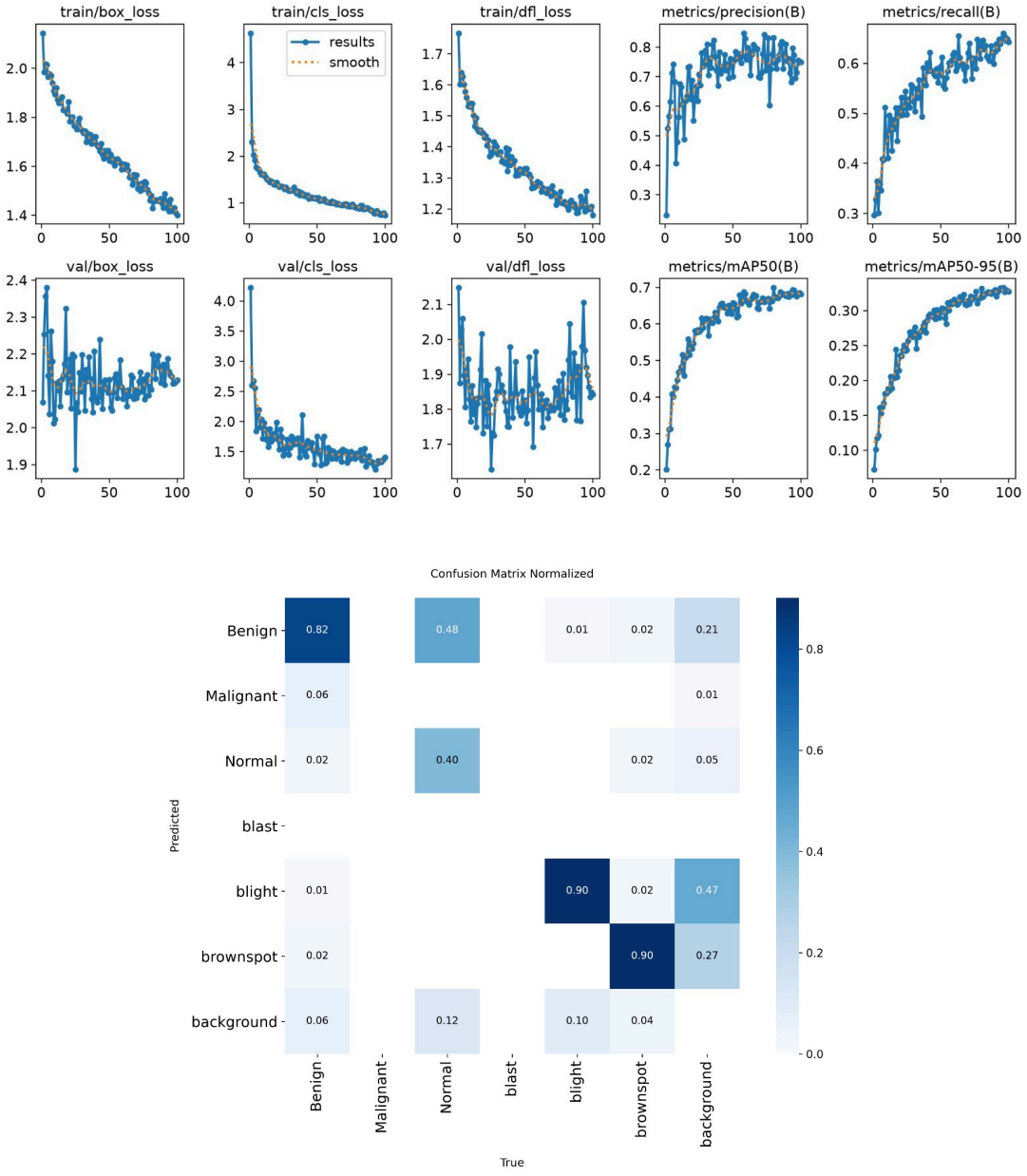


Figure 4.2: Adafruit IO Dashboard

4.2 PERFORMANCE METRICS

AI subsystem (rice disease detection)

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
All classes(Blast, Blight, Brownspot)	0.74	0.64	0.66	0.35



re 4.3: Model Training and Evaluation Results

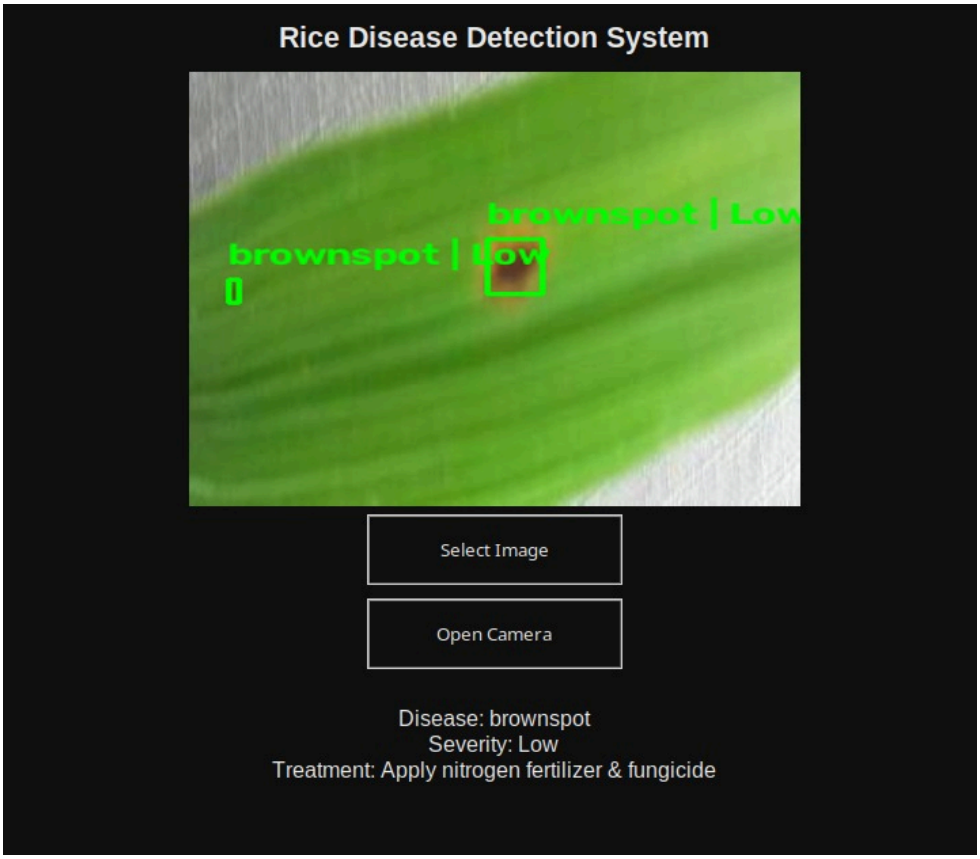
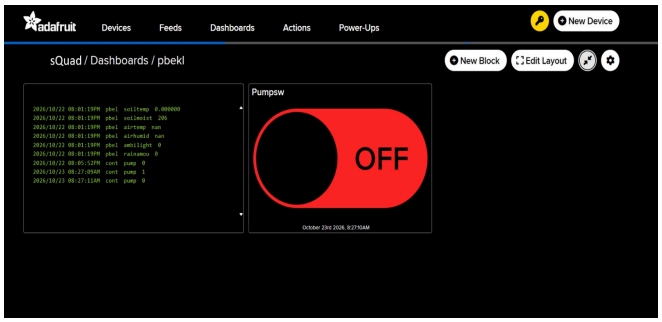
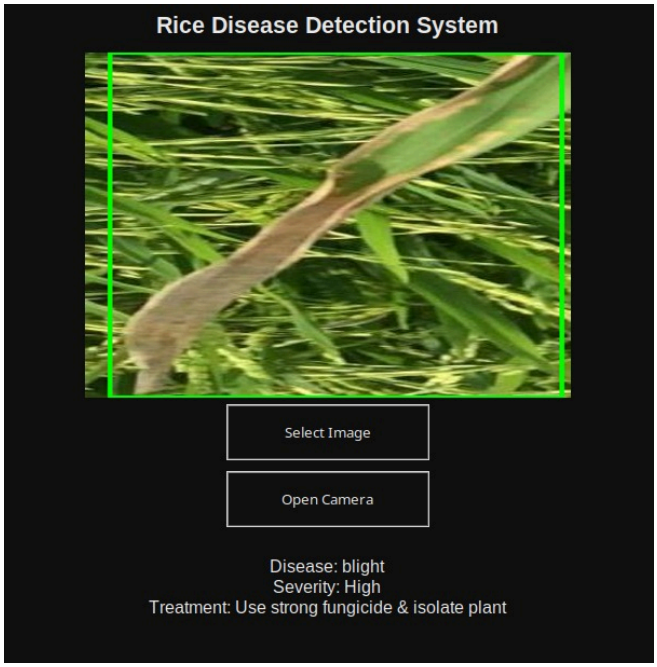


Figure 4.4: Rice Disease Detection Result

4.3 COMPARISON WITH EXPECTED RESULTS

Objective	Expected result	Observed result	Met ?
Send data frequently to Adafruit IO	Readings stored in feeds at the set interval		Yes

Objective	Expected result	Observed result	Met ?
Rice disease detection	Three diseases detected with good precision and recall		Yes
Automatic irrigation	Proposed originally	Not implemented – moved to future scope	No
AI + IoT integration	Proposed originally	Not implemented – moved to future scope	No

4.4 OBSERVATIONS AND ANALYSIS

1. The ESP32 provided enough analog channels and a stable Wi-Fi connection to read all five sensors and keep the Adafruit IO link running, which was not possible with the single analog input of the ESP8266 proposed earlier.
2. Adafruit IO gave a ready-made dashboard and history without building a custom server, but its data-rate limits restrict how often readings can be sent, so the upload interval has to be chosen carefully.
3. Pump control through the dashboard depends on internet availability and on the user giving the command; the system does not decide by itself when to irrigate.
4. Low-cost sensors such as the DHT11 and the resistive soil-moisture sensor have limited accuracy and drift, so calibration is needed.
5. The disease-detection model was trained on images from available datasets, so its performance on photographs taken in actual fields (different lighting, backgrounds and growth stages) may differ from the test-set results.

4.5 VALIDATION AND VERIFICATION WITH SUMMARY OF FINDINGS

Verification (was the system built as designed): the ESP32 reads all sensors, uploads them to the Adafruit IO feeds, and switches the pump through the relay on the user's dashboard command; the YOLO model was trained on the labelled dataset for the three disease classes.
Validation (does it meet the need): the dashboard lets the user watch field conditions and irrigate remotely, and the model detects the selected rice diseases from leaf images with the metrics given in Section 5.2.

Summary of findings: the IoT subsystem and the AI subsystem each work as intended on their own; automatic irrigation, ML on the collected IoT data and AI + IoT integration remain to be done.

5. FUTURE ENHANCEMENT

1. **Train an AI/ML model from the collected IoT data:** the sensor data stored in Adafruit IO can be used to train a model for irrigation prediction or crop-condition analysis, and can later enable automatic irrigation.
2. **Merge the disease-detection model with the IoT system:** show the detected disease and confidence on a common dashboard alongside the sensor data.
3. **Disease severity prediction and treatment recommendations** based on the detected disease, using reliable agricultural references.
4. **Automatic image capture** using a camera module such as the ESP32-CAM, sending images to a processing server for periodic disease monitoring.
5. **Automatic threshold-based irrigation** and support for multiple irrigation zones.
6. **Weather data, alerts and field-specific datasets** to improve the models for local conditions.

6. CONCLUSION & RECOMMENDATIONS

Conclusion

The Smart Rice Farming System shows how low-cost IoT hardware and a modern object-detection model can support rice farming. The IoT subsystem uses an ESP32 to read soil moisture, rain, temperature, humidity and light conditions, sends them frequently to Adafruit IO, and lets the user switch the irrigation pump ON or OFF remotely from the dashboard. The AI subsystem is a YOLO-based model trained on manually labelled images to detect Rice Blast, Rice Blight and Brown Spot from rice-leaf images.

The architecture changed from the original proposal: ESP8266 was replaced by ESP32, Adafruit IO was chosen as the cloud and dashboard platform, and irrigation became user-controlled instead of threshold-based. The project is delivered as two working parts, with their integration and data-driven ML left as future enhancements.

Recommendations

- Calibrate the soil-moisture sensor for the actual soil, and consider better sensors than the DHT11 for field use.
- Protect the electronics and the relay in a weatherproof enclosure and use a proper power supply for the pump.
- Keep the Adafruit IO key and Wi-Fi credentials secret and out of shared code.
- Add real field photographs to the disease dataset to improve reliability outside the lab.
- Use the model output only as decision support and confirm with an agricultural expert before treatment.

7. REFERENCES / BIBLIOGRAPHY

1.Jocher, G., Chaurasia, A., and Qiu, J., "Ultralytics YOLO," Ultralytics documentation. Available at: https://docs.ultralytics.com

2.Adafruit Industries, Adafruit IO documentation. Available at: https://io.adafruit.com

3.Adafruit Industries, Adafruit IO Arduino Library. Available at: https://github.com/adafruit/Adafruit_IO_Arduino

4.Espressif Systems, ESP32 technical documentation and datasheet. Available at: https://www.espressif.com/en/products/socs/esp32

5.DHT11, LM35, LDR, soil moisture sensor and rain sensor technical documentation / datasheets.

6.Research literature on rice diseases (Blast, Blight and Brown Spot) and their detection.

7.https://ieeexplore.ieee.org/document/10701486

8.https://ieeexplore.ieee.org/document/11081847

9.https://ieeexplore.ieee.org/document/9716089

10.https://link.springer.com/article/10.1038/s41598-026-62682-1

11.https://link.springer.com/article/10.1007/s42729-024-01629-9

12.https://www.mdpi.com/4019300

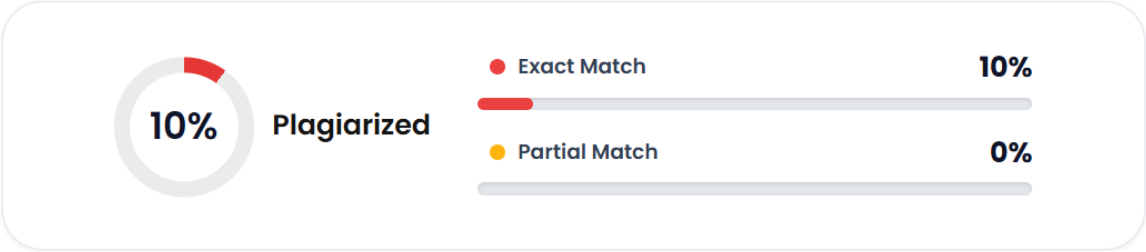
13.https://www.mdpi.com/4018430

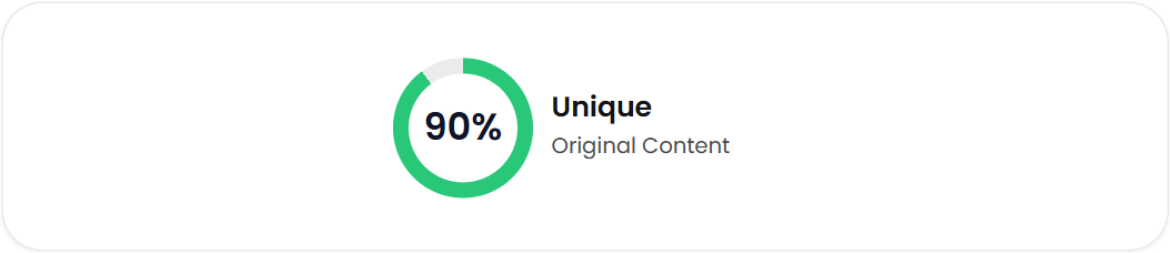
14.https://www.sciencedirect.com/science/article/pii/S2352484722013543

15.https://www.sciencedirect.com/science/article/pii/S2772375526000900

16.Plagiarism report by DupliChecker website

PLAGIARISM REPORT





APPENDIX: SAMPLE SOURCE CODE

A.1 ESP32 Firmware (Sensor Upload and Pump Control)

```
#include <WiFi.h>
#include <DHT.h>
#include "AdafruitIO_WiFi.h"
#define DHTPIN 13
#define DHTTYPE DHT11
#define lm 12
#define ldr 14
#define somoi 33
#define rain 2
#define pumpin 18
// Wi-Fi credentials
#define WIFI_SSID "ENTER_WIFI_NAME"
#define WIFI_PASS "ENTER_WIFI_PASSWORD"

// Adafruit IO credentials
#define IO_USERNAME  "ENTER_USERNAME_HERE"
#define IO_KEY       "aio_ENTER_IO_KEY_HERE"
long last=millis();

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
DHT dht(DHTPIN, DHTTYPE);

// Feed name
AdafruitIO_Feed *airtem = io.feed("pbel.airtemp");
AdafruitIO_Feed *airmoi = io.feed("pbel.airhumid");
AdafruitIO_Feed *soitem = io.feed("pbel.soiltemp");
AdafruitIO_Feed *soimoi = io.feed("pbel.soilmoist");
AdafruitIO_Feed *light = io.feed("pbel.ambilight");
AdafruitIO_Feed *rai = io.feed("pbel.rainamou");
AdafruitIO_Feed *pum = io.feed("cont.pump");
void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("DHT11 Sensor Ready!");
  Serial.println("Connecting to Adafruit IO");
  io.connect();
  pum->onMessage(handleMessage);
  while (io.status() < AIO_CONNECTED) {
    Serial.println("                -------\n\
               /       \\ \n\
              / O   O   \\ \n\
             |   <       | \n\
              \\  .___,  / \n\
               \\_______/ \n\
                \\    / \n\
           _______)  (_____ \n\
```

```
          /                \\");
    delay(200);
    Serial.println("                 -------\n\
                /       \\ \n\
               /  O   O  \\ \n\
              |     <     | \n\
               \\   .___, / \n\
                \\_______/ \n\
                 \\     / \n\
            _______)   (_____ \n\
          /                \\");
    delay(200);
    Serial.println("                 -------\n\
                /       \\ \n\
               /   O   O \\ \n\
              |      >    | \n\
               \\     .___ / \n\
                \\_______/ \n\
                 \\     / \n\
            _______)   (_____ \n\
          /                \\");
    delay(200);
    Serial.println("                 -------\n\
                /       \\ \n\
               /   -   - \\ \n\
              |      >    | \n\
               \\     .___ / \n\
                \\_______/ \n\
                 \\     / \n\
            _______)   (_____ \n\
          /                \\");
    delay(200);
    Serial.println("                 -------\n\
                /       \\ \n\
               /  o   o  \\ \n\
              |     >     | \n\
               \\   .___, / \n\
                \\_______/ \n\
                 \\     / \n\
            _______)   (_____ \n\
          /                \\");
    delay(200);
  }

  Serial.println("\nConnected to Adafruit IO!");
  pum->get();
}

void loop() {
```

```
  io.run();
  if(millis()-last>60000){
    Serial.print("WiFi status: ");
    Serial.println(WiFi.status());
    Serial.print("AIO status: ");
    Serial.println(io.statusText());
    last=millis();
    int lmval = analogRead(lm);
    float vol = lmval * (3.3 / 4095.0);
    float tem = vol * 100.0;
    Serial.print("Temperature: ");
    Serial.print(tem);
    Serial.println("°C");

    // Send data to the feed
    soitem->save(tem);
    float airhumi = dht.readHumidity();
    airmoi->save(airhumi);
    float airtemp = dht.readTemperature();
    airtem->save(airtemp);
    int raini=analogRead(rain);
    rai->save(raini);
    int soil=analogRead(somoi);
    soimoi->save(soil);
    int ligh=analogRead(ldr);
    light->save(ligh);
  }
  delay(10);
}
void handleMessage(AdafruitIO_Data *data) {
  int pin=data->toInt();
  digitalWrite(pumpin,pin);
  Serial.print("pump = ");
  Serial.println(pin);
}
```

A.2 YOLO Rice Disease Model (Training and Detection)

```
# rice_disease.yaml
# path: datasets/rice_disease
# train: images/train
# val:   images/val
# test:  images/test
# names: {0: Rice_Blast, 1: Rice_Blight, 2: Brown_Spot}

from ultralytics import YOLO

# 1. Load a pre-trained YOLO model and train on the labelled
rice-leaf dataset
model = YOLO("yolov8n.pt")
model.train(data="rice_disease.yaml", epochs=100, imgsz=640,
batch=16)

# 2. Evaluate (precision, recall, mAP) on the validation / test
split
metrics = model.val()
print("mAP50:", metrics.box.map50, " mAP50-95:",
metrics.box.map)

# 3. Run detection on a new rice-leaf image
results = model.predict("test_leaf.jpg", conf=0.25, save=True)
for r in results:
    for b in r.boxes:
        print(model.names[int(b.cls)], float(b.conf))
```